VOLUME I · EXAM CHEATSHEET · STUDY GUIDE

CMS 704 — Computer Architecture

PGD Computer Science, Rivers State University, Nkpolu-Oroworukwo, Port Harcourt · built from the lecturer's 14 pages of class notes, the lecturer's extra notes on memory and performance, and the official course outline

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Wednesday 05 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer **Units:** 3

READ THIS FIRST — HOW THIS PAPER DIFFERS FROM CMS 702

There is no past paper on file for CMS 704, so priorities come from what the lecturer actually spent board time on. Two things follow.

1. **This paper has calculation in it.** Number-base conversion, boolean minimization, K-maps, signed-number representation and the CPU performance equation are all **workable** questions with a right answer. Unlike a pure bookwork paper, these are where marks are won and lost — practise the method until it is automatic, and **always show the working line by line**, because method marks survive an arithmetic slip.
2. **The definitions still matter.** Architecture vs organization, cache, locality, RTL and performance were dictated almost word-for-word — those are the bookwork half.

The seven topics most likely to carry the paper: architecture vs organization (§2) · logic gates and truth tables (§3) · boolean minimization and K-maps (§5–§6) · RTL and micro-operations (§7) · number-base conversion (§8) · signed number representation (§9) · memory hierarchy, cache and locality (§10). Performance and the Iron Law (§11) is the newest material — freshly issued notes are a common source of exam questions.

1 Definitions to write word-for-word

TERM	DEFINITION
Computer architecture	Also known as the **Instruction Set Architecture (ISA)**. The set of rules, methods and procedures that define the **functionality, organization and implementation** of a computer system as seen by the machine-language programmer or compiler writer. It is the **contract between software and hardware** — a specification that remains constant across implementations.
Computer organization	The **actual implementation** of a computer architecture: how the functional units are constructed, interconnected and controlled to realize the specified architecture, and the engineering decisions affecting cost/performance, power consumption and physical size.
Logic gate	The fundamental building block of digital circuits — a device that performs a **boolean function**, a logical operation on one or more binary inputs producing a **single binary output**.
Logic expression minimization	Simplifying boolean equations to **reduce the hardware required** for a digital circuit; it cuts production cost, minimizes heat generation and improves speed.
Karnaugh map	A **visual, graphical matrix method** used to simplify boolean expressions of up to 4–6 variables, in which groupings are made in **powers of 2^n** to find the minimal sum of products.
Majority vote circuit	A digital circuit in which the **majority of the inputs determines the output**.
RTL	**Register Transfer Language** — a symbolic notation used to describe the internal organization, data flow and execution of **micro-operations between hardware registers**. It bridges high-level architectural design and concrete logic-gate implementation.
Micro-operation	An **elementary operation performed on data in CPU registers during a clock pulse** — arithmetic, logic or shift.
Control function	A **boolean condition that dictates when a transfer occurs**; written before a colon, e.g. `P: R2 ← R1`.

TERM	DEFINITION
Memory hierarchy	An **organizational pyramid that categorizes data storage by speed, cost and capacity**, so the processor can access frequently used data at very high speed without sacrificing the massive capacity needed for a user's whole library of files.
Cache memory	A **high-speed, volatile hardware component situated between the CPU and main memory (RAM)**, storing temporary copies of frequently accessed data and instructions so the processor retrieves them instantly, reducing latency and improving performance.
Cache hit / miss	**Hit** — the CPU checks the cache first and the data is found, so it is retrieved almost instantly (nanoseconds). **Miss** — the data is not in the cache, so the CPU must fetch it from the slower RAM, and it is then copied into the cache for future use.
Locality of reference	A program's **tendency to repeatedly access the same memory locations, or nearby ones, over a short period**. It is the foundation of cache memory design.
Performance	A measure of **how quickly and efficiently a computer system executes a given workload**, defined by **speed** (how fast a task completes) and **throughput** (how much work is done in a given time). Mathematically the **reciprocal of execution time**.
Sign bit	The bit used to represent whether a number is positive or negative, since a digital circuit has only 0s and 1s and no "−" symbol. It is the **MSB**: 1 = negative, 0 = positive.
Unsigned number	A binary number that **does not allow negative values**; an n-bit unsigned number has **2^n unique codes** and a range of **0 to $2^n - 1$**.

2 Architecture vs Organization — the guaranteed question

Both lists were dictated in full. A question asking you to **differentiate architecture from organization** is answered by defining both, giving the table, then quoting a few items from each list.

BASIS	COMPUTER ARCHITECTURE (ISA)	COMPUTER ORGANIZATION
What it answers	**What** the machine does — its behaviour as seen by a program	**How** the machine does it — how the units are built and wired
Level	Abstract **specification** / interface	Concrete **implementation**
Visible to	The machine-language programmer and the compiler writer	The hardware designer / engineer
Stability	**Constant** across different implementations of the same architecture	**Varies** between models that share one architecture
Driven by	Functional requirements — the software contract	Engineering trade-offs: cost/performance, power, physical size
Example	x86-64 — software compiled for it runs on **any** processor implementing it	Two x86-64 chips with different cache sizes, pipeline depths and clock speeds

ARCHITECTURE SPECIFIES (7)

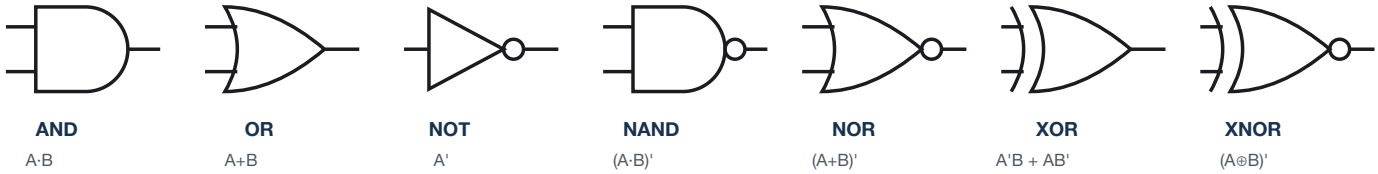
1. **The instruction set** — the operations the processor can execute (ADD, SUB, LOAD …).
2. **Data types and size** — 8-bit byte, 16-bit word, 32-bit double word.
3. **Register set** — the number, names and functions of programmer-visible registers.
4. **Addressing modes** — how instructions specify addresses: immediate, direct, indirect, indexed.
5. **Memory addressing model** — byte addressable or word addressable.

ORGANIZATION SPECIFIES (7)

1. **Data path design** — layout and width of buses, number of ALUs, register file ports.
2. **Control unit implementation** — hard-wired vs micro-programmed control.
3. **Pipelining structure** — number of stages, hazard handling.
4. **Memory hierarchy implementation** — cache size (L1, L2, L3), associativity, replacement policy, TLB size.
5. **Clock frequency and voltage scaling** — how fast the circuit runs, at what power cost.
6. **Parallelism mechanisms** — superscalar issue width, out-of-order execution, simultaneous multi-threading (SMT).

6. **Interrupt and exception handling** — how the processor responds to external events or internal faults.
7. **Input/output model** — memory-mapped I/O or special I/O instructions.
7. **Physical implementation** — transistor technology, chip area, thermal design power (TDP).

3 Logic gates — the seven you must be able to draw



The bubble (small circle) on the output is what turns AND into NAND, OR into NOR and XOR into XNOR — it *is* the NOT.

THE MASTER TRUTH TABLE — MEMORISE THIS BLOCK

A	B	A·B AND	A+B OR	(A·B)' NAND	(A+B)' NOR	A⊕B XOR	(A⊕B)' XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

The NOT gate has its own two-row table: `0 → 1`, `1 → 0`.

THE ONE-LINE RULE FOR EACH GATE

- **AND** — output is 1 **if and only if both inputs are 1**, else 0.
- **OR** — output is 0 **if both inputs are 0**, else 1.
- **NOT** — gives the **opposite** of the input.
- **NAND** — Not AND: `C = (A·B)'`.
- **NOR** — Not OR: `C = (A+B)'`.
- **XOR** — output 1 **where the inputs are different**.
- **XNOR** — output 1 **only when the inputs are the same**.

NUMBER OF ROWS IN A TRUTH TABLE

```
rows = 2ⁿ, where n = number of inputs
```

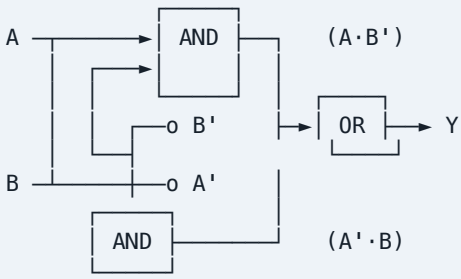
1 input → 2 rows · 2 inputs → 4 rows · **3 inputs → 8 rows**. Fill the input columns by counting up in binary — 000, 001, 010, 011, 100, 101, 110, 111 — so no combination is missed.

XOR built from the primary gates DRAWN IN CLASS

The exclusive-OR is not primitive — it is assembled from two NOTs, two ANDs and one OR, which is exactly what its sum-of-products expression says:

```
Y = A'B + AB'
```

Read the expression straight off the truth table: take every row where the output is 1, write the product of the inputs (complemented where the input is 0), and OR those products together. That procedure is the **sum of products (SOP)**, and it works for any truth table.



4 Boolean algebra — the seven laws

LAW	AND FORM	OR FORM	WHAT IT DOES FOR YOU
Identity	`A·1 = A`	`A+0 = A`	Removes a redundant constant
Null	`A·0 = 0`	`A+1 = 1`	Collapses a whole term to a constant
Idempotent	`A·A = A`	`A+A = A`	Deletes a duplicated literal
Complement	`A·A' = 0`	`A+A' = 1`	**The workhorse of minimization** — this is what makes grouping work
Absorption	`A·(A+B) = A`	`A + A·B = A`	Swallows the longer term entirely
Distributive	`A·(B+C) = A·B + A·C`	`A + B·C = (A+B)(A+C)`	Factors a common variable out of two terms
De Morgan	`(A·B)' = A' + B'`	`(A+B)' = A'·B'`	Pushes a bar through a bracket — **break the bar, change the sign**

The proof technique the notes insist on: to prove an identity such as `A·1 = A`, **build the truth table**. Anywhere a **1** appears as a column head it is 1 (true) all the way down that column, and a **0** is 0 (false) all the way down. Then compare the result column with the column for `A`: if they are identical, the identity holds.

```
A | 1 | A·1 | 0 | A+0        A·1 matches A   ✓
--|---|-----|---|----        A+0 matches A   ✓
0 | 1 |  0  | 0 |  0
1 | 1 |  1  | 0 |  1
```

5 Logic expression minimization

WHY MINIMIZE — THREE MARKS, THREE POINTS

1. **Hardware efficiency** — uses fewer logic gates and interconnections.
2. **Improved performance** — minimizes propagation delay, giving faster circuit operation.
3. **Cost and power** — lower power consumption and a smaller silicon chip footprint.

Opening sentence: *minimization simplifies boolean equations to reduce the hardware required for digital circuits; it cuts production cost, minimizes heat generation and improves speed.*

THE THREE TECHNIQUES

1. **Boolean algebra** — applying the mathematical laws to factor out and eliminate terms algebraically.
2. **Karnaugh map (K-map)** — a visual, graphical matrix method for up to 4–6 variables; groupings in powers of 2^n give the minimal sum of products.
3. **Quine–McCluskey algorithm** — a tabular algorithmic method for a large number of variables, or for computer-aided design.

Worked example — the majority vote circuit DONE IN CLASS

A **majority vote circuit** outputs 1 when the majority of its inputs are 1. With three inputs, that means any two or more.

#	A	B	C	F	MINTERM
0	0	0	0	0	
1	0	0	1	0	
2	0	1	0	0	
3	0	1	1	**1**	A'BC ①
4	1	0	0	0	
5	1	0	1	**1**	AB'C ②
6	1	1	0	**1**	ABC' ③
7	1	1	1	**1**	ABC ④

```
F = A'BC + AB'C + ABC' + ABC

Groupings — pair terms differing in ONE variable:

① and ④  ⇒  A'BC + ABC  ⇒  BC(A'+A) = BC
③ and ④  ⇒  ABC' + ABC  ⇒  AB(C'+C) = AB
② and ④  ⇒  AB'C + ABC  ⇒  AC(B'+B) = AC

∴  F = BC + AB + AC
```

Why this is legal: every grouping uses the complement law `X + X' = 1` together with the identity law `Y·1 = Y`. Note that minterm ④ is reused in all three groupings — **a minterm may belong to more than one group**, because `X + X = X` (idempotent). Four 3-input AND gates and one OR gate become three 2-input AND gates and one OR gate.

6 Karnaugh maps — the method that saves time

HOW TO BUILD AND READ A 3-VARIABLE K-MAP

1. Put **A** down the side and **BC** across the top.
2. Label the columns **00, 01, 11, 10** — **not** 00, 01, 10, 11. Adjacent columns must differ in exactly one bit (Gray code); this is the whole point of the map.
3. Write the output of each truth-table row into its cell.
4. Circle the 1s in **groups whose size is a power of 2** — 1, 2, 4, 8. Groups may overlap and may wrap around the edges.
5. For each group, **cancel the variable that changes** inside it and keep the ones that stay constant. That product is one term of the answer.
6. OR the terms together — that is the minimal sum of products.

The rule in one line, as written in the notes: *cancel the changing parameter.* A group of 2 kills one variable, a group of 4 kills two, a group of 8 kills three.

THE CELL LAYOUT

```
        BC
A \   00   01   11   10
   ┌────┬────┬────┬────┐
 0 │  0 │  1 │  3 │  2 │   ← decimal row numbers
   ├────┼────┼────┼────┤
 1 │  4 │  5 │  7 │  6 │
   └────┴────┴────┴────┘

Class result:   F = AB' + BC'
```

GROUP SIZE → WHAT SURVIVES

GROUP SIZE	VARIABLES CANCELLED	TERM LENGTH (3-VARIABLE MAP)
1 cell	0	3 literals, e.g. A'BC
2 cells	1	2 literals, e.g. BC
4 cells	2	1 literal, e.g. A
8 cells	3	The function is simply 1

Always make the groups as large as legally possible — a bigger group means a shorter term and fewer gates. Then check every 1 is covered by at least one group.

7 Register Transfer Language

RTL is a symbolic notation used in computer architecture to describe the **internal organization, data flow and execution of micro-operations between hardware registers**. It bridges the gap between high-level architectural design and concrete logic-gate implementation, allowing engineers to concisely map out how data moves through a CPU during each clock cycle.

SYNTAX AND SYMBOLS

SYMBOL	MEANING
Capital letters	Denote specific hardware registers — `R1`, `R2`, `PC`, `MAR`
Arrow ←	A **unidirectional transfer** from the source on the right to the destination on the left
Parenthesis ( )	A distinct sub-part or individual bits of a register — `R1(0–7)` is the lower 8 bits
Colon :	Terminates a **control function** — the operation executes only if the preceding condition is true
Comma ,	Separates micro-operations that execute **simultaneously** in a single clock cycle

BUS NOTATION

```
Bus ← R1     R1 drives the system bus
R2  ← Bus    R2 latches data from the bus
```

THE FOUR TYPES OF MICRO-OPERATION

TYPE	WHAT IT DOES	EXAMPLE
Register transfer	Moves binary data from one processor register to another **without modifying** it	`R2 ← R1`
Arithmetic	Numeric operations — addition, subtraction, increment — on data inside the registers	`R3 ← R1 + R2`
Logic	**Bitwise** operations (AND, OR, XOR, NOT) manipulating bits independently within the register	`R ← R1 ∧ R2`
Control function	Boolean conditions dictating **when** a transfer occurs	`P: R2 ← R1`

Micro-operations are elementary operations performed on data in CPU registers during a clock pulse — arithmetic, logic and shift.

Translating English into RTL — the exam skill

STATEMENT	RTL
If `K = 1` then transfer R1 into R2	`K: R2 ← R1`
If `K = 1` then `R2 ← R1`, otherwise `R2 ← R4`	`K: R2 ← R1 · K': R2 ← R4` — the else branch uses the **complement** of the control signal
If `Z = 1` AND `C = 0` then increment the program counter	`Z·C': PC ← PC + 1` — AND of conditions is written as a **product**
Transfer R1 to R2 and R3 to R4 in the same clock cycle	`R2 ← R1, R4 ← R3` — the comma means **simultaneously**
Load the lower 8 bits of R1 into R2	`R2 ← R1(0–7)`

The PC keeps track of the next instruction to be fetched by the CPU from memory. The **fetch/execute cycle** is: **Fetch** (PC supplies the address) → **Decode** (the control unit interprets the instruction) → **Execute**.

8 Number systems and conversion

THE POSITIONAL SYSTEM

BASE	NAME	DIGITS
10	Decimal / denary	0 1 2 3 4 5 6 7 8 9
8	Octal	0 1 2 3 4 5 6 7
2	Binary	0 1
16	Hexadecimal	0–9 then A B C D E F (A = 10 … F = 15)

$4832_{10} = 4\times10^3 + 8\times10^2 + 3\times10^1 + 2\times10^0$
$21673_8 = 2\times8^4 + 1\times8^3 + 6\times8^2 + 7\times8^1 + 3\times8^0$
$10111001_2 = 1\times2^7 + 1\times2^5 + 1\times2^4 + 1\times2^3 + 1\times2^0$

Every base works the same way: the digit is multiplied by the base raised to its position, counting from **0** at the right-hand end. Fractions continue with negative powers: 8^{-1}, 8^{-2}, 8^{-3} …

BASE 10 → ANY BASE: SUCCESSIVE DIVISION

```
4832 to base 8
 8|4832
 8| 604 – 0
 8|  75 – 4
 8|   9 – 3
 8|   1 – 1
  |   0 – 1

read UP: 11340₈
```

```
4832 to base 16
16|4832
16| 302 – 0
16|  18 – 14 (E)
16|   1 –  2
  |   0 –  1

read UP: 12E0₁₆
```

Divide by the target base, write the **remainder** beside each line, stop at 0, then **read the remainders upwards**. For hexadecimal, convert remainders 10–15 into A–F before writing them down.

The four conversions of 4832 — learn this one number

BASE	4832 BECOMES	CHECK BY EXPANDING IT BACK TO BASE 10
2	1001011100000_2	$2^{12} + 2^9 + 2^7 + 2^6 + 2^5 = 4096 + 512 + 128 + 64 + 32 =$ **4832** ✓
8	11340_8	$1\times4096 + 1\times512 + 3\times64 + 4\times8 + 0 =$ **4832** ✓
16	$12E0_{16}$	$1\times4096 + 2\times256 + 14\times16 + 0 = 4096 + 512 + 224 =$ **4832** ✓
fraction	11340.0012_8	$= 4832.00244140625_{10}$ — see the fraction method below

FRACTIONS: REPEATED MULTIPLICATION BY THE BASE

```
Convert 0.00244140625 to base 8

0.00244140625 × 8 = 0.01953125  → 0
0.01953125    × 8 = 0.15625     → 0
0.15625       × 8 = 1.25        → 1
0.25          × 8 = 2.0         → 2   stop

∴ 0.00244140625₁₀ = 0.0012₈
```

Multiply the fraction by the base, take the **integer part** as the next digit, carry the fractional remainder forward, and read the digits **downwards** — the opposite direction to the whole-number method.

THE SHORTCUT CONVERSIONS — NEVER DIVIDE FOR THESE

$8 = 2^3 \rightarrow 3 : 1 \quad \cdot \quad 16 = 2^4 \rightarrow 4 : 1$

```
Octal ↔ binary — group in THREES from the right
4362₈  =  100 011 110 010  =  100011110010₂

1001011100000₂ → 1 001 011 100 000 → 11340₈

Hex ↔ binary — group in FOURS from the right
1001011100000₂ → 0001 0010 1110 0000
                   1    2    E    0  → 12E0₁₆
```

Pad the leftmost group with leading zeros. Octal and hexadecimal exist precisely because they are **compact shorthands for binary** — this is why the conversion is pure grouping and needs no arithmetic.

BINARY ADDITION RULES

```
0 + 0     = 0
0 + 1     = 1
1 + 0     = 1
1 + 1     = 10   (0 carry 1)
1 + 1 + 1 = 11   (1 carry 1)

   10111₂   (23)
 + 11101₂   (29)
 ─────────
  110100₂   (52) ✓
```

OCTAL SUBTRACTION

```
   246₈      (166)
 − 127₈      ( 87)
 ─────       
   117₈      ( 79) ✓
```

When you borrow in base 8 you borrow **8**, not 10. Check the answer by converting all three numbers to decimal.

OCTAL MULTIPLICATION

```
    336₈      (222)
  × 227₈      (151)
  ──────
   3022       336₈ × 7₈
   674        336₈ × 2₈  ←shift
 + 674        336₈ × 2₈  ←shift
 ──────
 101362₈      (33522) ✓
```

In base 8, carry whenever a column reaches 8 — divide by 8, write the remainder, carry the quotient.

9 Signed and unsigned numbers

In a real digital circuit there is **no "−" symbol** — the hardware only has 0s and 1s. So one bit, the **most significant bit (MSB)**, is used to carry the sign. A number that never goes negative is **unsigned**; one that may go negative is **signed**. The same bit pattern means different values under different schemes, so you must always state which scheme is in use.

UNSIGNED NUMBERS

`unique codes = 2ⁿ  ·  range = 0 … 2ⁿ − 1`

N	UNIQUE CODES	RANGE
4	16	0 … 15
8	256	0 … 255
16	65 536	0 … 65 535

A 4-bit number has 16 codes but its largest value is **15** — because one code is spent on **0**. That is exactly why the maximum is `2ⁿ − 1` and not `2ⁿ`. It is the single most common slip on this topic.

THE THREE SIGNED REPRESENTATIONS AT A GLANCE (4-BIT)

VALUE	SIGNED MAGNITUDE	ONE'S COMPLEMENT	TWO'S COMPLEMENT
+3	0011	0011	0011
−3	**1011**	**1100**	**1101**
+7	0111	0111	0111
−7	**1111**	**1000**	**1001**
+0	0000	0000	0000
−0	**1000**	**1111**	**0000**

Positive numbers are **identical** in all three schemes. Note the last row: signed magnitude and one's complement both have **two representations of zero**; two's complement has only one — which is the main reason real hardware uses it.

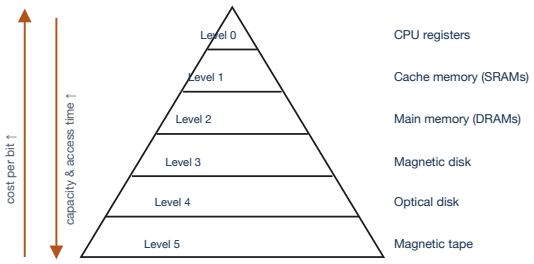
SCHEME	RULE FOR A NEGATIVE NUMBER	WORKED: −56 (56_{10} = 0111000_2)
Signed magnitude	Write the magnitude in binary, then **set the MSB to 1**. The remaining n−1 bits are preserved and hold the magnitude.	`0111000` → change the first digit 0 to 1 → **`1111000`₂**
One's complement	The MSB represents the sign; **invert every bit** of the positive form (negative numbers only).	`0111000` → negate → **`1000111`₂**
Two's complement	**Invert every bit, then add 1.** The MSB is the sign bit.	`0111000` → `1000111` → +1 → **`1001000`₂**

THE SECOND WORKED EXAMPLE FROM THE NOTES — −127 (127_{10} = 01111111_2)

Signed magnitude: `11111111₂` · **One's complement:** `10000000₂` · **Two's complement:** `10000001₂` (invert to `10000000`, then add 1).

Encoding schemes to name if asked: BCD (Binary Coded Decimal) · **EBCDIC** (Extended Binary Coded Decimal Interchange Code) · **ASCII**.

10 Memory hierarchy, cache and locality



Up the pyramid: higher cost per bit, faster access, smaller capacity. Down: cheaper per bit, slower, far larger.

The sentence that earns the top mark: the goal of the hierarchy is to **create the illusion that the entire massive storage pool is as fast as the topmost level**, bridging the gap between ultra-fast, expensive, limited CPU registers and the slow, cheap, massive capacity of hard drives.

LEVEL	SPEED	CAPACITY	PURPOSE
CPU registers	Extremely fast (sub-nanosecond)	Tiny (bytes; 16–64 bits)	Built inside the CPU core to hold the exact data being executed in the current cycle
Cache (L1/L2/L3)	Very fast (SRAM)	Small–moderate (KB–MB)	Buffer between registers and system RAM; uses locality to preload frequently and recently used data
Main memory (RAM)	Moderate	Moderate–large (16–64 GB)	Primary working memory for the OS and running applications
Secondary storage	Slow (milliseconds)	Massive (terabytes)	**Non-volatile** SSDs and HDDs; retains data when powered down; long-term repository
Tertiary storage	Very slow	Infinite (scalable)	Offline, archival and backup data — tape libraries, optical disks

Cache memory

Cache memory is a high-speed, volatile hardware component **situated between the CPU and main memory (RAM)**. It stores temporary copies of frequently accessed data and instructions so the processor can retrieve them instantly, significantly **reducing processing latency and improving system performance**. It works because of **locality of reference**.

LEVEL	LOCATION	SIZE	SPEED
L1	Directly on the CPU core	2 KB – 64 KB per core	**Fastest**, smallest
L2	Inside or immediately outside the core	256 KB – 512 KB	Bridge for the L1 cache
L3	Shared among all cores	1 MB – 8 MB or higher	**Largest but slowest**; feeds into L2

CACHE MEMORY MAPPING — THREE CONFIGURATIONS

MAPPING	HOW A MEMORY BLOCK IS PLACED
Direct mapped	Each block is mapped to **exactly one** cache location.
Fully associative	Similar in structure, but a memory block may be mapped to **any** cache location rather than a pre-specified one.
Set associative	A **compromise** between the two: each block maps to a **subset** of cache locations. Also called **N-way set associative** — a main-memory location may be cached to any of N locations in the L1 cache.

Cache hit — the CPU checks the cache first and finds the data, retrieving it in nanoseconds. **Cache miss** — the data is absent, so the CPU fetches from slower RAM and the block is then copied into the cache for future use.
Other caching: disk/file cache (the OS holds recently read files in RAM) and **browser cache** (images, scripts and page data kept in local storage).

DATA WRITING POLICIES

POLICY	MECHANISM	CONSEQUENCE
Write-through	Data written to **both** the cache and main memory at the same time	More writing, so **latency upfront**, but memory stays consistent
Write-back	Data written to the **cache only** at first; it may reach main memory later, but need not, and this does not inhibit the interaction	**More efficient**, but data may be **inconsistent** between cache and main memory

Locality of reference

Locality of reference (the **principle of locality**) describes a program's tendency to **repeatedly access the same memory locations, or nearby ones, over a short period**. It is the foundation for cache memory design, allowing CPUs to fetch frequently used data ahead of time and dramatically increasing system speed.

1. TEMPORAL LOCALITY — LOCALITY IN TIME

If a specific memory location is accessed, it is **highly likely to be accessed again in the near future**.
Example: a **loop or a counter** — the CPU continuously accesses the same variables and instructions within that short time frame.

2. SPATIAL LOCALITY — LOCALITY IN SPACE

If a specific memory location is accessed, **nearby locations are also likely to be accessed soon**.
Example: arrays or sequential data structures — the processor brings in a **block of adjacent data** in anticipation of the next request.

Other forms of locality: sequential locality · branch locality.

11 Performance and the Iron Law

In computer architecture, **performance measures how quickly and efficiently a computer system executes a given workload or program**. It is primarily defined by **speed** — how fast a task completes — and **throughput** — how much work is done in a given time.

THE TWO CORE METRICS

1. **Response time (execution time)** — the total time required to complete a **single task** from start to finish.
2. **Throughput (bandwidth)** — the total amount of work completed **per unit of time**, e.g. instructions or programs executed per second.

QUANTITATIVE MEASUREMENT

```
Performance = 1 / Execution Time
```

```
Relative Performance = Perf A / Perf B = Exec Time B / Exec Time A
```

THE IRON LAW OF PROCESSOR PERFORMANCE

```
CPU Time = Instruction Count × CPI × Clock Cycle Time
```

FACTOR	DEFINITION	DETERMINED BY
Instruction Count (IC)	Total number of instructions executed in a program	The software **compiler** and the **ISA**

FACTOR	DEFINITION	DETERMINED BY
Cycles Per Instruction (CPI)	Average number of clock cycles to execute one instruction	**Hardware design** and pipeline efficiency
Clock Cycle Time	Duration of a single clock tick — the **inverse of clock frequency** (e.g. 3.0 GHz)	**Semiconductor technology** and hardware engineering

The law is "iron" because **all three factors multiply** — improving one while worsening another may leave performance unchanged. A compiler that halves IC but doubles CPI wins nothing.

Worked example 1 — CPU time. A program executes **2×10^9 instructions** with **CPI = 1.5** on a **3.0 GHz** processor.

```
Clock cycle time = 1 / 3.0×10⁹ = 0.333 ns

CPU Time = 2×10⁹ × 1.5 × 0.333×10⁻⁹ s
         = 1.0 second
```

Worked example 2 — relative performance. Machine A runs a program in **10 s**, Machine B in **15 s**.

```
Relative Performance = Exec B / Exec A
                     = 15 / 10 = 1.5

∴ Machine A is 1.5 times faster than Machine B
```

Worked example 3 — MIPS. Same program as example 1:

```
MIPS = IC / (Execution Time × 10⁶)
     = 2×10⁹ / (1.0 × 10⁶)
     = 2000 MIPS
```

Other performance indicators: MIPS — Million Instructions Per Second · **FLOPS** — Floating Point Operations Per Second.

12 Things you can lose easy marks on

1. Labelling K-map columns **00, 01, 10, 11**. They must be **00, 01, 11, 10** — adjacent cells differ in one bit only.
2. Saying an n-bit unsigned number reaches **2^n**. It reaches **$2^n - 1$**; one code is spent on zero.
3. Mixing up architecture and organization. Architecture = **what/specification**; organization = **how/implementation**.
4. Writing `R2 → R1` when you mean `R2 ← R1`. The arrow points **to the destination**.
5. Forgetting the **complement** in an RTL else-branch: it is `K': R2 ← R4`, not `K`.
6. Grouping octal into **fours** — octal is **threes** ($8 = 2^3$); fours are for hexadecimal ($16 = 2^4$).
7. Reading division remainders **downwards**. Read them **upwards**; fractions are the ones read downwards.
8. Confusing one's and two's complement. Two's complement is one's complement **plus 1**.
9. Saying L3 is the fastest cache. L1 is **fastest and smallest**; L3 is **largest and slowest**.
10. Swapping temporal and spatial locality. **Temporal = time = same location again** (a loop); **spatial = space = nearby locations** (an array).
11. Writing throughput where response time is asked. **Response time = one task end to end**; **throughput = work per unit time**.
12. Quoting the Iron Law with a division. It is a **product** of all three factors.

TIMING PLAN FOR A 3-HOUR PAPER

Spend the first 5 minutes reading everything and marking which questions are **calculations** — do those first, while your arithmetic is sharp and before fatigue causes slips. Budget roughly **30 minutes per question**, and inside a calculation always write the **formula first**, then substitute, then evaluate: a wrong final number with the right formula and substitution still collects most of the marks, while a bare answer collects none. For minimization questions, **draw the truth table even when it is not asked for** — it is the only reliable way to catch a missed minterm, and it doubles as working.